TRACERA — Engineering Specification & Single Source of Truth

Version: 1.0 — Pre-implementation final baseline

Date: 11 September 2026

Project: Tracera

Maintainers: Shrey + Arman

Status: Approved for implementation

This document is the engineering source of truth for Tracera. It is intentionally practical rather than promotional. If the implementation changes the architecture, API, data model, ML pipeline, or security boundary, update this document in the same pull request.

---

0. START HERE

0.1 What is Tracera?

Tracera is a defensive website analysis and risk-assessment platform for developers and startups.

A user submits a website URL. Tracera safely observes the publicly accessible response and extracts structured signals from HTTP, TLS, cookies, HTML, and basic performance characteristics. A deterministic analysis engine identifies concrete configuration and quality issues. A risk engine aggregates those signals into an overall score. In later phases, a trained ML model may learn a validated risk/severity relationship from a versioned dataset.

Tracera is not a penetration-testing framework and does not perform exploitation, authentication bypass, brute force, payload injection, or intrusive vulnerability testing.

0.2 The core principle

Tracera has three distinct layers:

1. **Deterministic analysis** — use rules when the answer is directly measurable.
2. **Machine learning** — use ML only when learning from multiple signals provides value that a transparent rule cannot.
3. **Language generation** — use an LLM later, if useful, only to explain already-verified findings.

The project must never become “an LLM scans your website.”

0.3 MVP

The MVP is deliberately small:

• Single URL

• One safe HTTP fetch

• SSRF-hardened URL validation

• HTTP/TLS/cookie/HTML feature extraction

• 10–15 deterministic findings

• Evidence attached to every finding

• Fixed, transparent risk score

• PostgreSQL persistence

• Minimal Next.js dashboard

• FastAPI backend

• Deployed frontend/backend

No ML is required for the first working MVP.

That is intentional. A bad model trained on weak labels is worse than a transparent baseline.

0.4 ML responsibility — Shrey

Shrey owns:

• Feature schema design

• Feature engineering

• Dataset pipeline

• Label strategy

• ML experiments

• Baseline models

• Model evaluation

• Risk-model integration

• Model versioning

• ML inference

• Statistical analysis

• ML documentation

Shrey also contributes to backend architecture and security decisions.

0.5 Full-stack responsibility — Arman

Arman owns:

• Next.js frontend

• Dashboard and report UI

• API integration

• Scan submission flow

• Scan status/result presentation

• Database integration where appropriate

• Authentication in V1+

• Projects/history UI

• Deployment integration

• Frontend testing

• Shared API contract implementation

Arman also contributes to backend integration and scanner work. Ownership does not mean isolation: both developers review architectural changes.

0.6 Shared responsibility

Both:

• Architecture

• API contracts

• Security

• Scanner boundaries

• Finding schema

• Database design

• Testing

• Deployment

• Code review

• Documentation

• Final product decisions

0.7 First implementation task

Create the repository and FastAPI skeleton.

The first backend endpoint should:

```
POST /api/v1/scans
    ↓
validate URL
    ↓
SSRF guard
    ↓
safe HTTP fetch
    ↓
return status + selected response metadata
```

Do not begin with authentication, ML, a complex dashboard, Redis, Celery, Kubernetes, or a multi-page crawler.

---

1. PRODUCT DEFINITION

1.1 Problem

Developers can deploy websites with insecure or poor configurations without realizing what an external observer can see.

Relevant issues include:

- Missing security headers
- Weak cookie configuration
- HTTPS/TLS problems
- Information disclosure
- Unsafe CORS configurations
- Mixed content
- Basic structural/performance issues
- Disclosed software versions associated with known CVEs

Existing tools often specialize in one dimension. Tracera's goal is to combine several passive signals into one evidence-backed developer report while keeping detection transparent and adding ML only where it is genuinely useful.

1.2 Target users

Primary:

- Developers
- Student developers
- Indie hackers
- Startups
- Small engineering teams
- Technical founders

Secondary:

- QA engineers
- DevOps engineers
- Security-conscious agencies
- Consultants

1.3 User promise

After entering a URL, the user should be able to answer:

- What is wrong?
- How serious is it?
- What evidence supports the finding?
- Why does it matter?
- What should I fix first?

1.4 Explicit non-goals

Tracera does not initially:

• Exploit vulnerabilities

• Attempt login bypass

• Brute-force credentials

• Inject attack payloads

• Scan arbitrary internal networks

• Scan authenticated areas

• Crawl the entire internet

• Replace a professional penetration test

• Claim comprehensive vulnerability coverage

• Claim that an ML score equals real-world exploit probability

---

2. PRODUCT SCOPE

2.1 MVP

Feature	MVP
URL validation	Yes
SSRF protection	Yes
Single HTTP fetch	Yes
Redirect handling	Limited + revalidated
TLS metadata	Yes
Security headers	Yes
Cookie flags	Yes
Basic HTML analysis	Yes
Basic performance proxies	Yes
Deterministic findings	Yes
Evidence	Yes
Fixed risk score	Yes
Database	Yes
Dashboard	Yes
Authentication	No
ML model	No
CVE correlation	No
LLM	No
Multi-page crawling	No
Browser rendering	No
Background queue	No

2.2 V1

• ML risk/severity model, only if it passes the ML go/no-go criteria

• Supabase authentication

• Scan history

• Projects

• CVE correlation for disclosed software versions

• Improved report UI

• Background execution if scan latency requires it

• Dataset/model version metadata

2.3 V2

• Anomaly detection

• Multi-page scanning

• Scan comparison

• Trend analysis

• Optional LLM explanation layer

• Continuous monitoring

• CI/CD integration only if the core product is already stable

---

3. SYSTEM ARCHITECTURE

3.1 High-level architecture

```
flowchart TD
    A[User] --> B[Next.js Frontend]
    B --> C[FastAPI API]
    C --> D[URL Validator + SSRF Guard]
    D --> E[Safe Collector]
    E --> F[Feature Extraction]
    F --> G[Deterministic Rule Engine]
    F --> H[ML Risk Layer - V1+]
    F --> I[CVE Correlation - V1+]
    G --> J[Finding Aggregator]
    H --> J
    I --> J
    J --> K[Risk Scoring]
    K --> L[Recommendation Generator]
    L --> M[(PostgreSQL)]
    M --> B
```

3.2 Important architectural correction

Do not create separate “security engine,” “performance engine,” “SEO engine,” and “technical engine” services for the MVP.

Most of these checks are simply rules operating on the same feature vector.

Use:

```
Feature Vector
      ↓
Rule Engine
      ↓
Categorized Findings
```

The ML layer consumes the feature vector independently.

3.3 Component responsibilities

Component	Does	Does not do
URL Validator	Validate scheme, hostname, DNS/IP safety	Fetch
Collector	Safe HTTP request, response capture	Decide severity
Feature Extractor	Raw response → versioned features	Generate findings
Rule Engine	Features → deterministic findings	Train models
ML Layer	Features → learned score/prediction	Detect every individual issue
CVE Correlator	Disclosed software → known CVEs	Claim complete vulnerability coverage
Risk Engine	Combine findings/model outputs	Crawl sites
Report Generator	Structured result → report	Perform detection
API	Orchestrate and expose services	Contain business logic everywhere
Frontend	User interaction and visualization	Perform scanning

---

4. DATA FLOW

```
URL
 ↓
Canonicalization
 ↓
SSRF validation
 ↓
DNS/IP validation
 ↓
Safe request
 ↓
Raw response
 ■■■ headers
 ■■■ status
 ■■■ timing
 ■■■ TLS metadata
 ■■■ cookies
 ■■■ HTML
 ↓
Feature extraction
 ↓
Versioned feature vector
 ■■■ Rule engine
 ■■■ ML model (V1+)
 ■■■ CVE lookup (V1+)
 ↓
Findings
 ↓
Risk aggregation
 ↓
Recommendations
 ↓
Persist
 ↓
Frontend report
```

Every stage must have clear input/output boundaries.

---

5. SCANNER AND SECURITY ARCHITECTURE

5.1 Threat model

The highest-risk component is not the scanned website. It is Tracera itself because the server accepts user-controlled URLs and makes outbound requests.

The primary threat is SSRF.

5.2 SSRF requirements

Before any network request:

1. Only allow `http` and `https`.
2. Reject `file`, `ftp`, `gopher`, and other schemes.
3. Resolve the hostname.
4. Validate resolved addresses.
5. Block loopback.
6. Block private ranges.
7. Block link-local ranges.
8. Block multicast/reserved ranges where appropriate.
9. Do not send cloud credentials or authentication headers.
10. Apply strict timeout and response-size limits.

Redirects must not bypass these controls.

Do not simply enable unrestricted automatic redirects.

For each redirect:

```
redirect URL
    ↓
canonicalize
    ↓
resolve
    ↓
validate IP
    ↓
allow/reject
```

Limit redirect count.

5.3 DNS rebinding

Hostname validation followed by a normal hostname connection can create a time-of-check/time-of-use problem.

The collector must use a design that prevents a malicious hostname from resolving to a safe address during validation and an internal address during connection.

This is a launch-blocking security requirement.

5.4 Outbound request policy

MVP requests are:

• Anonymous

• Read-only

• GET only

• No credentials

• No cookies copied from the user

• No arbitrary custom headers from users

• Bounded by timeout

• Bounded by response size

• Bounded by redirect count

5.5 Abuse prevention

Implement:

• Per-IP rate limiting

• Scan concurrency limits

• URL validation

• Request timeouts

• Response-size limits

• Logging

• Abuse/error monitoring

Do not build a distributed rate limiter for the MVP.

5.6 Privacy

Prefer storing:

• URL

• selected response metadata

• extracted features

• findings

• scores

• timestamps

Avoid storing full response bodies unless absolutely required.

If a body is temporarily stored for parsing, treat it as transient data.

---

6. FEATURE ENGINEERING

6.1 Feature schema

Every live feature vector must include:

```
feature_schema_version
```

Training and inference must use the same schema contract.

A feature added or removed in a breaking way requires a schema-version change.

6.2 Feature groups

HTTP

• Status code

• Response size

• Response time

• Content type

• Redirect count

• HTTPS enabled

Security headers

• HSTS

• CSP

• X-Content-Type-Options

• X-Frame-Options

• Referrer-Policy

• Permissions-Policy

Cookies

• Secure

• HttpOnly

• SameSite

• Number of cookies

• Third-party cookie indicators where observable

TLS

• Certificate expiry status

• TLS version metadata where safely obtainable

• Certificate subject/issuer metadata where useful

Do not claim parity with dedicated TLS analyzers.

HTML / structure

• HTML size

• Script count

• Stylesheet count

• Image count

• Iframe count

• Form count

• External domain count

• HTTP resources on HTTPS page

Technology disclosure

• Server header presence

• X-Powered-By presence

• Framework/version indicators when explicitly disclosed

6.3 Feature quality rules

Features must be:

• Deterministic where possible

• Reproducible

• Documented

• Typed

• Versioned

• Testable

Do not create meaningless features simply to increase feature count.

---

7. DETERMINISTIC RULE ENGINE

7.1 Why rules exist

A missing header is not a machine-learning problem.

For example:

```
HSTS header exists?
    yes → pass
    no  → finding
```

The rule engine should produce findings with:

• Rule ID

• Category

• Severity

• Title

• Evidence

• Explanation

• Recommendation

• Detection source

• Confidence

7.2 Initial rule candidates

Examples:

Rule	Category	Initial severity
HTTP instead of HTTPS	Security	High
Missing HSTS	Security	Medium
Missing CSP	Security	Medium
Missing X-Content-Type-Options	Security	Low
Missing X-Frame-Options	Security	Medium
Missing Referrer-Policy	Security	Low
Missing Permissions-Policy	Security	Low
Insecure cookie	Security	High
HttpOnly missing on sensitive cookie	Security	Medium
SameSite missing/weak	Security	Medium
Server version disclosure	Configuration	Low
X-Powered-By disclosure	Configuration	Low
Wildcard CORS with credentials	Security	High
Mixed content	Security	High
Excessive response size	Performance	Low

These are candidates, not final truth. Severity must be reviewed against authoritative security guidance before implementation.

7.3 Rule contract

Conceptually:

```
Finding = {
    "rule_id": "...",
    "category": "...",
    "severity": "...",
    "title": "...",
    "description": "...",
    "evidence": {...},
    "recommendation": "...",
    "source": "rule_engine"
}
```

7.4 Rule testing

Every rule requires:

• Positive test

• Negative test

• Edge-case test where applicable

Malformed headers must be tested.

---

8. ML PROBLEM DEFINITION

8.1 The biggest ML problem

The project does not begin with a trustworthy labeled dataset saying:

“This website has these features and its true real-world exploitation risk is 73/100.”

Therefore, do not pretend that the model predicts actual probability of compromise.

The realistic supervised ML problem is:

Learn a validated mapping from a website's observed feature/finding vector to an expert-defined risk/severity target, and test whether the learned model generalizes better than a transparent fixed baseline.

This is a narrower but defensible applied-ML problem.

8.2 ML go/no-go rule

The ML model ships only if it demonstrates meaningful improvement over the fixed baseline on a held-out evaluation set.

If it merely reproduces the rule formula:

do not market it as an intelligent risk model.

That is a critical project rule.

8.3 Candidate ML tasks

Primary: risk-tier classification

Input:

```
feature vector
```

Output:

```
Low / Medium / High / Critical
```

Secondary: continuous risk regression

Input:

```
feature vector
```

Output:

```
0–100 risk score
```

This should only be used if the target is sufficiently defensible.

Future: anomaly detection

Use an unsupervised model to identify unusual feature combinations relative to a corpus.

This is a better fit for anomaly detection than pretending anomalies are vulnerabilities.

---

9. DATASET STRATEGY

9.1 Data sources

Potential sources:

1. Sites owned/controlled by the team
2. Explicitly consented sites
3. Public aggregated website datasets
4. Synthetic feature vectors
5. Public vulnerability metadata for deterministic CVE correlation

Do not mass-scan arbitrary third-party websites for training data.

9.2 Label hierarchy

Each sample must record label provenance.

```
label_source =
    expert_review
    rule_derived
    synthetic
    external_dataset
```

Do not mix these silently.

9.3 Weak supervision

Rule-derived labels may bootstrap the first model.

However, this creates a major risk:

```
rules
 ↓
labels
 ↓
model
```

The model may simply learn to reproduce the rules.

Therefore, evaluation must include:

• Held-out examples
• Expert-reviewed samples
• Alternative test cases
• Comparison against the fixed baseline

9.4 Synthetic data

Synthetic data is acceptable for testing and bootstrapping, but must not be presented as real-world evidence.

Synthetic generation should vary:

• Header combinations
• Cookie configurations
• TLS states
• Finding combinations
• Noise
• Missing values
• Feature distributions

Train/test generation must prevent template leakage.

9.5 Data split

Avoid random row splitting when rows are highly correlated.

Prefer grouping by:

• Website
• Domain
• Synthetic generation family
• Dataset source

so similar samples do not appear in both train and test.

---

10. ML MODEL DEVELOPMENT

10.1 Baselines

Start with:

1. Fixed formula

2. Logistic Regression

3. Random Forest

4. Gradient boosting / XGBoost if justified

Do not start with a neural network.

10.2 Why classical ML first?

The expected dataset is small.

The feature space is structured/tabular.

Classical models provide:

• Fast experimentation

• Strong baselines

• Interpretability

• Low inference cost

• Simple deployment

10.3 Model comparison

For each experiment record:

```
model
dataset_version
feature_schema_version
hyperparameters
training_date
metrics
notes
artifact_version
```

10.4 Evaluation

For classification:

Primary:

• Macro F1

Secondary:

• Critical recall

• Per-class precision

• Per-class recall

• Confusion matrix

• PR-AUC where appropriate

Do not use accuracy as the headline metric for an imbalanced problem.

For regression:

• MAE

• RMSE

• Calibration/error analysis

---

11. RISK SCORING

11.1 MVP scoring

Use a transparent formula.

Conceptually:

```
raw_score =
    Σ severity_weight(finding)
```

Then normalize to:

```
0–100
```

The exact weights must be documented and tested rather than treated as scientific truth.

Suggested initial tiers:

```
0–24    Low
25–49   Medium
50–74   High
75–100  Critical
```

These are product tiers, not formal security standards.

11.2 V1 ML scoring

The ML model may replace or augment the fixed formula only after passing the ML go/no-go gate.

The UI must show whether the score was produced by:

```
fixed_baseline
ml_model
fallback_baseline
```

Never silently fall back and pretend the model generated the score.

11.3 CVE scores

If a known CVE is correlated, use the authoritative CVE metadata rather than inventing a new CVE severity.

Tracera must say:

Known CVE associated with disclosed software/version.

It must not say:

This website is vulnerable.

unless the available evidence actually justifies that statement.

---

12. CVE CORRELATION

CVE correlation is deterministic lookup, not ML.

Conceptual flow:

```
Disclosed technology/version
        ↓
Normalize fingerprint
        ↓
Query vulnerability database
        ↓
Known CVEs
        ↓
Attach evidence + limitations
```

Limitations:

• Many sites do not disclose versions.

• Disclosed version strings may be inaccurate.

• A known CVE does not prove exploitability of the specific deployment.

• Absence of a CVE does not prove safety.

This feature must be described conservatively.

---

13. LLM POLICY

An LLM is optional and post-MVP.

It may eventually:

• Summarize findings

• Explain impact

• Rewrite recommendations for developers

• Generate a report narrative

It must not be the authoritative detector.

Correct:

```
Scanner
 → rules/ML
 → verified findings
 → LLM explanation
```

Incorrect:

```
Scanner
 → LLM
 → "probably vulnerable"
```

---

14. BACKEND ARCHITECTURE

14.1 Recommended stack

• Python 3.12

• FastAPI

• Pydantic

• httpx

• BeautifulSoup

• PostgreSQL

• SQLAlchemy

• Alembic

• scikit-learn

• XGBoost if justified

• pytest

14.2 Directory structure

```
tracera/
■■■ backend/
■   ■■■ app/
■   ■   ■■■ api/
■   ■   ■   ■■■ v1/
■   ■   ■■■ core/
■   ■   ■■■ collector/
■   ■   ■■■ features/
■   ■   ■■■ rules/
■   ■   ■■■ scoring/
■   ■   ■■■ ml/
■   ■   ■■■ cve/
■   ■   ■■■ reports/
■   ■   ■■■ db/
■   ■   ■■■ main.py
■   ■■■ tests/
■
■■■ frontend/
■   ■■■ app/
■   ■■■ components/
■   ■■■ lib/
■   ■■■ tests/
■
■■■ ml/
■   ■■■ data/
■   ■■■ notebooks/
■   ■■■ preprocessing/
■   ■■■ features/
■   ■■■ training/
■   ■■■ evaluation/
■   ■■■ models/
■   ■■■ experiments/
■
■■■ docs/
■■■ scripts/
■■■ .env.example
■■■ .gitignore
■■■ README.md
```

14.3 Separation rule

The ML training environment should not become tightly coupled to the web application.

Training code lives in ml/.

Inference code lives in:

```
backend/app/ml/
```

---

15. API DESIGN

Base path:

```
/api/v1
```

15.1 Create scan

```
POST /api/v1/scans
```

Request:

```
{
  "url": "https://example.com"
}
```

Response:

```
{
  "scan_id": "uuid",
  "status": "completed"
}
```

For MVP, this may remain synchronous if scan duration is acceptable.

15.2 Get scan

```
GET /api/v1/scans/{scan_id}
```

15.3 Get findings

```
GET /api/v1/scans/{scan_id}/findings
```

15.4 Get report

```
GET /api/v1/scans/{scan_id}/report
```

15.5 V1 project endpoints

```
GET /api/v1/projects
POST /api/v1/projects
GET /api/v1/projects/{project_id}
```

API contracts must be written before frontend/backend integration.

---

16. FINDING SCHEMA

Canonical finding:

```
{
  "id": "uuid",
  "rule_id": "SEC-HSTS-001",
  "category": "security",
  "severity": "medium",
  "title": "HSTS header is missing",
  "description": "The response does not include Strict-Transport-Security.",
  "evidence": {
    "header": "Strict-Transport-Security",
    "observed": false
  },
  "impact": "Browsers are not instructed to enforce HTTPS for future requests.",
  "recommendation": "Configure Strict-Transport-Security after confirming HTTPS is correctly deployed.",
  "source": "rule_engine",
  "confidence": 1.0
}
```

ML-derived metadata must be stored separately from deterministic evidence.

---

17. DATABASE

17.1 MVP tables

scans

```
id
url
canonical_url
status
feature_schema_version
risk_score
risk_tier
score_source
created_at
completed_at
error_code
```

findings

```
id
scan_id
rule_id
category
severity
title
description
evidence_json
recommendation
source
confidence
created_at
```

17.2 V1 tables

```
projects
model_versions
users/auth references
```

Authentication identity should be delegated to Supabase/Auth rather than duplicating credential storage.

---

18. FRONTEND ARCHITECTURE

18.1 Core pages

Home / Scan

• URL input

• Authorized-use notice

• Scan button

• Validation feedback

Scan Results

• Overall risk score

• Risk tier

• Finding counts

• Categories

• Prioritized findings

Finding Detail

• Title

• Severity

• Evidence

• Impact

• Recommendation

• Detection source

History — V1

• Previous scans

• Score trend

• Scan date

• URL

18.2 UX principle

The dashboard should answer:

“What should I fix first?”

not merely:

“Here are 15 technical measurements.”

---

19. TEAM WORKFLOW — SHREY + ARMAN

19.1 Git strategy

One repository:

```
github.com/<owner>/Tracera
```

Branches:

```
main
feature/shrey-ml-baseline
feature/shrey-feature-extraction
feature/arman-dashboard
feature/arman-api-integration
feature/shared-risk-engine
```

Do not work directly on main.

19.2 Workflow

```
Issue
 ↓
Feature branch
 ↓
Implementation
 ↓
Tests
 ↓
Commit
 ↓
Pull Request
 ↓
Other person reviews
 ↓
Merge
 ↓
main
```

19.3 Ownership

Area	Owner	Reviewer
ML/data	Shrey	Arman
Feature engineering	Shrey	Arman
Frontend	Arman	Shrey
API integration	Arman	Shrey

Area	Owner	Reviewer
Scanner	Shared	Other
Security	Shared	Other
DB	Arman/Shared	Shrey
Risk scoring	Shrey	Arman
Deployment	Shared	Other

19.4 Interface-first development

Before implementing connected components, agree on:

• Request schema
• Response schema
• Finding schema
• Feature schema
• Error schema

This is more important than having identical local environments.

---

20. TESTING STRATEGY

20.1 SSRF

Test:

• localhost
• loopback IPv4
• loopback IPv6
• private IPv4
• link-local
• metadata endpoint ranges
• redirects to private addresses
• invalid schemes
• DNS edge cases

20.2 Collector

Mock network requests in CI.

CI must not scan arbitrary external sites.

20.3 Feature extractor

Use saved response fixtures.

Test:

• normal HTML
• malformed HTML
• missing headers
• multiple cookies
• large responses
• redirects

20.4 Rule engine

Every rule gets unit tests.

20.5 Risk engine

Pure-function tests.

20.6 ML

Track:

• dataset version
• feature schema version
• model version
• metrics
• confusion matrix

20.7 Frontend

Test:

• loading state
• error state
• empty findings
• critical findings
• API failures
• long reports

---

21. DEPLOYMENT

21.1 MVP

```
Next.js
   ↓
Vercel

FastAPI
   ↓
Railway

PostgreSQL
   ↓
Supabase
```

This is intentionally simple.

21.2 No premature infrastructure

Do not add:

• Kubernetes

• Docker Swarm

• Kafka

• Celery

• Redis

• separate ML server

unless an actual requirement appears.

21.3 Queue trigger

Introduce a background queue when one or more becomes true:

• Multi-page crawling ships

• Median scan time becomes several seconds

• Production requests time out

• Concurrent scans materially degrade API reliability

Until then, synchronous execution is acceptable for MVP.

---

22. MLOPS

Keep MLOps proportional to project size.

Required:

• Dataset version

• Feature schema version

• Model version

• Reproducible training script

• Evaluation report

• Artifact metadata

Not initially required:

• Full model registry platform

• Kubernetes model serving

• Complex orchestration

• Enterprise monitoring stack

Example model metadata:

```
{
  "model_version": "risk-v1.0",
  "dataset_version": "dataset-v0.3",
  "feature_schema_version": "features-v1",
  "algorithm": "xgboost",
  "trained_at": "2026-09-11",
  "macro_f1": 0.0,
  "critical_recall": 0.0
}
```

The values above are placeholders, not performance claims.

---

23. DEVELOPMENT ROADMAP

Phase 0 — Architecture

Deliverables:

• This document

• GitHub repository

• Issue board

• Branch rules

• API contract draft

• Finding schema

Done when:

• Shrey and Arman agree on the pipeline.

Phase 1 — Safe scanner

Build:

• URL validation

• SSRF guard

• Collector

• Basic API

Done when:

• Valid public URLs work.
• Known SSRF test cases fail safely.
• Timeouts and response caps work.

Phase 2 — Feature extraction

Build:

• HTTP features
• TLS metadata
• cookie parser
• HTML parser
• schema version

Done when:

• Same fixture produces deterministic features.

Phase 3 — Rule engine

Build:

• 10–15 rules
• Evidence
• Recommendations
• Unit tests

Done when:

• Every rule has tests.

Phase 4 — Database + dashboard

Build:

• Scan persistence
• Findings persistence
• Basic UI

Done when:

```
URL → scan → report
```

works end-to-end.

Phase 5 — Dataset

Build:

• Dataset schema

• Synthetic generator

• Public-data ingestion where feasible

• Label provenance

• Split strategy

Done when:

• Dataset can be regenerated.

Phase 6 — ML baseline

Build:

• Fixed baseline

• Logistic regression

• Tree-based baseline

• Evaluation pipeline

Done when:

• Results are reproducible.

Phase 7 — ML decision gate

Ask:

Does the model provide measurable value beyond the fixed baseline?

If no:

• Do not ship it as a core ML feature.

• Investigate better labels/features/problem formulation.

If yes:

• Continue to integration.

Phase 8 — ML integration

Build:

• Inference wrapper

• Model metadata

• Fallback baseline

• Monitoring/logging

Done when:

• Live inference matches offline preprocessing.

Phase 9 — CVE correlation

Build:

• Technology fingerprinting

• External vulnerability lookup

• Evidence and limitations

Done when:

• Known disclosed versions produce correct, conservative results.

Phase 10 — V1 polish

Build:

• Auth

• Projects

• History

• Better reports

• Scan comparison if justified

---

24. SUCCESS CRITERIA

System

• Safe URL handling

• Reliable scan completion

• Reproducible findings

• Stable API contracts

• P95 scan duration measured on a curated test set

ML

The model must:

• Beat or materially complement the fixed baseline

• Report macro F1

• Report Critical recall

• Include confusion matrix

• Use a leakage-resistant split

• Record dataset/model versions

Product

A user should understand a finding without needing a developer from the Tracera team to explain it.

---

25. TOP RISKS

1. **SSRF vulnerability** — launch blocker.
2. **Weak ML labels** — can make the model scientifically meaningless.
3. **Synthetic-data leakage** — can create fake high metrics.
4. **ML becoming cosmetic** — rule engine works and ML never ships meaningfully.
5. **Overclaiming security coverage** — damages credibility.
6. **Scope creep** — too many features before the core pipeline works.
7. **Scanner latency** — synchronous model eventually stops scaling.
8. **Feature-schema drift** — live inference no longer matches training.
9. **False positives** — users stop trusting findings.
10. **Unreviewed security assumptions** — a student scanner can accidentally become a security liability.

---

26. DEBUGGING PLAYBOOK

Frontend broken

Check:

1. API URL
2. Browser network tab
3. Response schema
4. CORS
5. Loading/error state

Backend broken

Check:

1. Request validation
2. Logs by scan ID
3. Pipeline stage
4. Exception
5. Response contract

Scanner broken

Check:

1. URL validation
2. DNS resolution
3. SSRF rejection
4. Timeout
5. Redirect
6. Response-size cap

Never weaken the SSRF guard just to make a URL work.

Features wrong

Check:

1. Fixture
2. Parser
3. Feature version
4. Header normalization
5. Edge cases

ML bad

Check in this order:

1. Labels
2. Dataset split
3. Feature schema
4. Class imbalance
5. Leakage
6. Thresholds
7. Model

Do not immediately switch to a more complex model.

High false positives

For deterministic rules, a false positive usually means:

The rule is wrong.

Fix the detection logic before adding probabilistic complexity.

Model metrics suspiciously high

Suspect:

• Leakage

• Duplicate domains

• Synthetic template overlap

• Rule-derived target leakage

• Preprocessing fitted before the split

---

27. ASSUMPTION REGISTER

Assumption	Risk	Validation
Single-fetch analysis is enough for MVP	Some JS-dependent signals are invisible	Compare against selected real sites
Rule-derived labels can bootstrap ML	Model may learn the rules	Expert-reviewed holdout
Classical ML is enough	Complex relationships may be missed	Compare baselines
Synchronous scanning is acceptable	Slow sites block requests	Measure P95
Two people can finish V1	UI can consume all time	Timebox phases
Public datasets are usable	Access/format/licensing constraints	Verify before committing
Disclosed versions enable useful CVE correlation	Most sites may not disclose versions	Measure coverage

---

28. DECISION LOG

Decision 1 — One repository

Chosen: one GitHub repository with feature branches.

Reason: simplest collaboration model for two developers while preserving review and history.

Decision 2 — No queue in MVP

Chosen: synchronous scan.

Reason: single-fetch scanning does not justify Redis/worker complexity initially.

Reconsider when: multi-page scanning or real latency/concurrency problems appear.

Decision 3 — No ML in MVP

Chosen: fixed risk formula first.

Reason: no trustworthy labels exist at project start. A rushed model would create fake sophistication.

Decision 4 — Classical ML first

Chosen: Logistic Regression → tree-based model.

Reason: structured/tabular features, small expected dataset, cheap inference.

Decision 5 — LLM is downstream

Chosen: optional explanation layer only after verified findings.

Reason: an LLM should not be responsible for determining whether a security finding exists.

Decision 6 — CVE correlation is deterministic

Chosen: external vulnerability lookup based on disclosed software/version.

Reason: this is database matching, not machine learning.

Decision 7 — Two deployed application services

Chosen: Next.js + FastAPI, with managed PostgreSQL.

Reason: sufficient for the team and MVP scale.

---

29. COMPETITIVE POSITIONING

Tracera should not claim to replace specialized security products.

It is better positioned as:

A developer-oriented passive website risk assessment platform that combines evidence-backed configuration analysis, structured risk aggregation, and—when validated—machine learning.

Existing categories already do individual pieces well:

• Security-header analyzers

• TLS analyzers

• Browser performance analyzers

• Vulnerability databases

• AI website auditors

Tracera's defensible project-level differentiation is the combination of multiple passive signals, transparent evidence, risk aggregation, and a measured ML layer.

That differentiation is narrow. That is acceptable.

---

30. RESUME VALUE

Strong technical claims if actually implemented

• Engineered structured features from HTTP/TLS/HTML responses.

• Built an SSRF-hardened server-side URL analysis pipeline.

• Developed deterministic security/configuration detection rules with evidence.

• Constructed a versioned dataset and evaluated classical ML models for risk classification.

• Evaluated imbalanced classification using macro-F1 and critical-tier recall.

• Deployed ML inference inside a FastAPI production service.

• Designed a deterministic + ML hybrid architecture.

Claims to avoid unless actually true

Do not write:

• “Built an AI penetration tester.”

• “Built an autonomous vulnerability scanner.”

• “AI detects all website vulnerabilities.”

• “ML predicts real-world exploit probability.”

• “Built enterprise MLOps.”

• “Used deep learning” if the final model is classical ML.

Technical honesty is part of the project's credibility.

---

31. FUTURE ROADMAP

Potential future work, in priority order:

1. Scan history

2. Scan comparison

3. Continuous monitoring

4. Better anomaly detection

5. Multi-page analysis

6. Browser-rendered analysis

7. GitHub integration

8. CI/CD integration

9. Repository/dependency analysis

10. Optional LLM explanation

Do not build all of these automatically.

Each requires a separate product/technical justification.

---

32. RESEARCH REQUIRED

These are not settled facts and must be verified before implementation:

• Best publicly available datasets for website/security configuration analysis

• Licensing and access conditions

• Practical public aggregate website datasets

• Authoritative severity guidance for each rule

• CVE/CPE matching reliability for disclosed versions

• Current API/rate-limit details of external vulnerability sources

• Robust DNS/IP handling strategy for SSRF-safe fetching

• Whether static HTTP fetching captures enough signal for the chosen MVP rules

• Whether browser rendering materially improves useful findings

• Whether the ML target is sufficiently independent from the deterministic rules

Never turn an assumption into a fact merely because it appears in an earlier version of this document.

---

33. IMPLEMENTATION CHECKLIST

Repository

• [ ] GitHub repo created

• [ ] Shrey added

• [ ] Arman added

• [ ] Branch protection configured

• [ ] README created

• [ ] `.gitignore`

• [ ] `.env.example`

Backend

• [ ] FastAPI
• [ ] URL validator
• [ ] SSRF guard
• [ ] Collector
• [ ] Feature extractor
• [ ] Rule engine
• [ ] Risk engine
• [ ] Tests

Database

• [ ] PostgreSQL
• [ ] Migrations
• [ ] scans
• [ ] findings

Frontend

• [ ] URL form
• [ ] Scan loading state
• [ ] Results page
• [ ] Finding cards
• [ ] Error states

ML

• [ ] Dataset schema
• [ ] Data generator
• [ ] Label provenance
• [ ] Baseline
• [ ] Logistic Regression
• [ ] Tree-based model
• [ ] Evaluation
• [ ] Model metadata
• [ ] Inference integration

Security

• [ ] SSRF tests

• [ ] Redirect revalidation

• [ ] DNS/IP controls

• [ ] Timeout

• [ ] Response-size cap

• [ ] Rate limiting

• [ ] No outbound credentials

• [ ] Authorized-use notice

Deployment

• [ ] Backend deployed

• [ ] Frontend deployed

• [ ] Database connected

• [ ] Environment variables

• [ ] Production smoke test

• [ ] Curated site validation

---

34. FIRST 10 IMPLEMENTATION TASKS

1. Create the `Tracera` GitHub repository and add Arman as collaborator.

2. Create `main` and branch-protection rules.

3. Create the repository directory structure.

4. Create FastAPI application skeleton.

5. Implement and unit-test the SSRF validation layer.

6. Implement the safe single-request collector.

7. Implement `POST /api/v1/scans`.

8. Implement the first versioned feature schema.

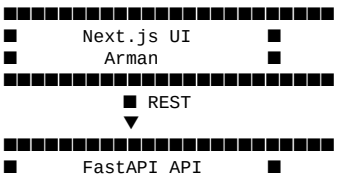
9. Implement the first 5 deterministic rules with tests.

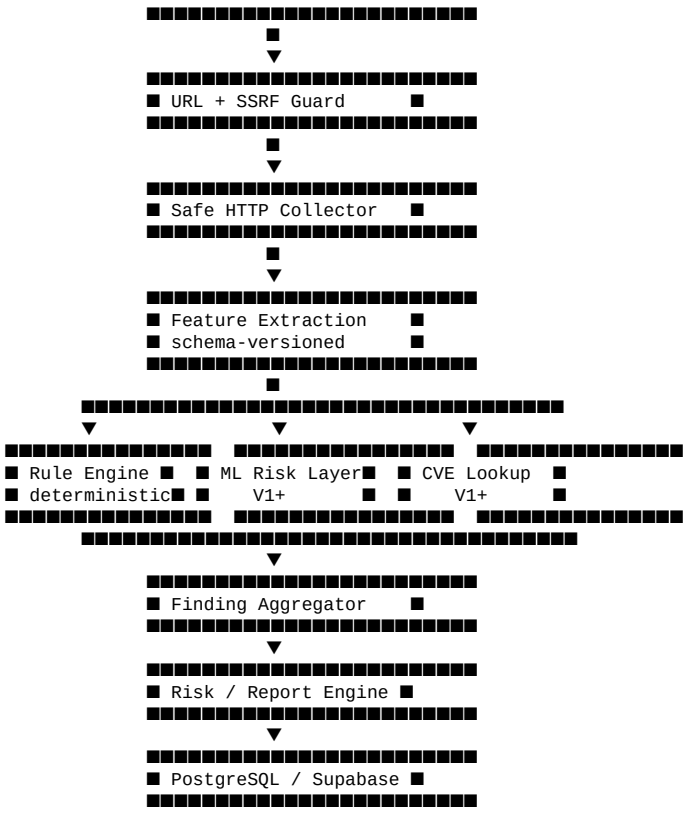
10. Run the first end-to-end scan against a curated set of authorized/public test sites and document the results.

---

35. FINAL AUTHORITATIVE ARCHITECTURE

This is the architecture to build unless a documented decision changes it.





Final stack

Layer	Technology
Frontend	Next.js
Backend	FastAPI
Language	Python 3.12
HTTP	httpx
HTML	BeautifulSoup
Database	PostgreSQL / Supabase
ORM	SQLAlchemy
Migrations	Alembic
ML	scikit-learn → XGBoost if justified
Testing	pytest
Frontend hosting	Vercel
Backend hosting	Railway
ML serving	In-process FastAPI
Queue	None in MVP
LLM	Optional V2
Browser automation	Not MVP

---

36. START HERE — DAY 1

Do exactly this:

1. Create GitHub repository

```
2. Add Arman
3. Clone repository
4. Create backend skeleton
5. Create SSRF test cases
6. Implement URL validation
7. Implement safe collector
8. Expose POST /api/v1/scans
9. Test against authorized/public targets
10. Document the observed behavior
```

Do not touch:

```
ML
LLM
authentication
multi-page crawling
fancy dashboard
Redis
Celery
Kubernetes
```

until the safe single-page scanning pipeline works.

The first objective is not to make Tracera impressive.

The first objective is to make this pipeline reliable:

```
URL
 ↓
SAFE FETCH
 ↓
FEATURES
 ↓
FINDINGS
 ↓
SCORE
 ↓
REPORT
```

Once that works, the ML work has a real system to improve rather than a theoretical project surrounding a model.

---

37. DOCUMENT MAINTENANCE RULE

Whenever a meaningful implementation decision changes:

1. Update the relevant architecture section.
2. Update the decision log.
3. Update API/database/schema documentation if affected.
4. Update the roadmap.
5. Make the documentation change in the same pull request as the code.

Tracera's code and documentation must not drift apart.

End of Tracera v1.0 Engineering Specification.